Reviewer Pack — Minimal Tropical Root Count and Circuit Monotone Width

Entry 162b03379794 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:05:24 UTC

Minimal Tropical Root Count and Circuit Monotone Width

Entry ID: `162b03379794` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 20:05:24 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every Boolean circuit C with n inputs, the minimal tropical root count (mtr(C)) of the associated tropical polynomial is linearly correlated with its monotone width w_mon(C), such that mtr(C) = O(w_mon(C)).

Rationale (proposer's reasoning):

> Tropical geometry has been used to study properties of max-plus semirings, which are closely related to circuit complexity. The minimal tropical root count measures the number of roots in the tropical setting, and it is believed that a lower bound on this count could provide insights into the structural properties of circuits.

Taxonomy category: `TROPICAL_GEOOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `46d5a6e5c42475f5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every Boolean circuit C with n inputs, if less than 80% of 30 random seeds yield mtr(C) within O(w_mon(C)), then the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `(tropical geometry AND Boolean circuit complexity) AND minimal tropical root count` - `monotone width IN tropical geometry AND Boolean circuit complexity` - `minimal tropical root count ~ monotone width IN Boolean circuits`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1812.08038v3] On refined count of rational tropical curves - [http://arxiv.org/abs/2309.17302v2] Geometry of tropical extensions of hyperfields - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        if n == 1:
            return ['0', '1']
        else:
            subcircuits = [generate_boolean_circuit(n // 2) for _ in range(2)]
            return [f'({sub[0]} OR {sub[1]})' for sub in zip(subcircuits[0], subcircuits[1])]

    def evaluate_circuit(circuit, inputs):
        if isinstance(circuit, str):
            return circuit
        else:
            left = evaluate_circuit(circuit[0], inputs)
            right = evaluate_circuit(circuit[2], inputs)
            return '1' if left == '1' or right == '1' else '0'

    def count_tropical_roots(circuit):
        n = len(circuit)
        roots = set()
        for i in range(2**n):
            inputs = [str((i >> j) & 1) for j in range(n)]
            if evaluate_circuit(circuit, inputs) == '1':
                roots.add(tuple(int(bit) for bit in inputs))
        return len(roots)
```

```
    def monotone_width(circuit):
        n = len(circuit)
        if n == 1:
            return 1
        else:
            left_width = monotone_width(circuit[0])
            right_width = monotone_width(circuit[2])
            return max(left_width, right_width) + 1

    n = random.randint(5, 40)
    circuit = generate_boolean_circuit(n)
    mtr_C = count_tropical_roots(circuit)
    w_mon_C = monotone_width(circuit)

    if mtr_C is None or w_mon_C is None:
        return {
            "metric_name": "Minimal Tropical Root Count and Circuit Monotone Width",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "Mapping undefined"
        }

    return {
        "metric_name": "Minimal Tropical Root Count and Circuit Monotone Width",
        "metric_value": mtr_C,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": mtr_C <= 2 * w_mon_C,  # Simplified for testing
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1000, 9999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results if res["metric_value"] is not N
    std_metric_value = (sum((res["metric_value"] - mean_metric_value)**2 for res in results if res
    support_fraction = sum(res["conjecture_holds"] for res in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample='Mapping undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4a2c7b.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4a2c7b.py", line 56, in run_trial
    mtr_C = count_tropical_roots(circuit)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4a2c7b.py", line 41, in count_tropica
    if evaluate_circuit(circuit, inputs) == '1':
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3f4a2c7b.py", line 33, in evaluate_circ
    right = evaluate_circuit(circuit[2], inputs)
                             ~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify if the conjecture holds for all circuits and seeds as required by the support condition. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13642
2	preregistration	ollama_remote	glm4:latest	0	0	13808
3	novelty	ollama_remote	glm4:latest	0	0	10499
4	novelty	ollama_remote	glm4:latest	0	0	10017
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31787
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13325
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9922
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10122
9	judge	ollama_remote	glm4:latest	0	0	11445

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 124568 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/162b03379794.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/162b03379794.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/162b03379794.tar.gz` (if generated)